AI Agents and Retrieval-Augmented Generation for Materials Simulation

Submitted by

XU Jiamin

Department of Mechanical Engineering
The University of Hong Kong

A dissertation submitted in partial fulfillment of the requirements for the Degree of Master of Science in Mechanical Engineering at the University of Hong Kong

Student Name:	**XU Jiamin**
University No.:	**3036316455**
Supervisor:	**WEN Tongqi**
Moderator or Co-supervisor:	
Date:	**2025/4/30**

ABSTRACT

While large language models (LLMs) have demonstrated impressive capabilities in general-domain natural language processing (NPL), their application to specialized technical fields such as materials science remains challenging. Conventional LLM architectures face significant limitations in processing domain-specific data structures and technical content, primarily due to their dependence on static training datasets that often result in outdated or inaccurate outputs. Recent advances in Retrieval-Augmented Generation (RAG) and AI agent present a transformative approach to address these constraints. This dissertation introduces an innovative framework that integrates RAG technology with LLMs to significantly enhance their performance in materials simulation tasks, overcoming both knowledge gap and output reliability challenges that have traditionally hindered their effective application in this domain.

In this dissertation, we first evaluate performance of several mainstream LLMs, including general-purpose models and domain-specific models, on code-based and doc-based datasets related to Pymatgen. According to the overall results, we select two representative models to be enhanced by RAG with output validation function and re-evaluate them on same datasets. This new RAG-enhanced architecture strengthens the capability of LLMs in materials simulation domain from two aspects: (1) Bridge the knowledge gap between LLMs inherent database and the specialized materials simulation applications by RAG augmentation; (2) Standardize the LLMs outputs by strict format verification. This approach establishes a foundation for developing more powerful LLM-based solutions in materials simulation. Furthermore, this modular framework shows great potential for broader scientific applications, as its adaptable design can be easily extended to other technical domains beyond materials simulation. All the scripts are developed based on Python in this dissertation.

Keywords: Large Language Model, Retrieval-Augmented Generation, AI agent, materials simulation

DECLARATION

I hereby declare that this dissertation represents my own work, except where due acknowledge is made, and that it has not been previously included in a thesis, dissertation or report submitted to this university or to any other institution for a degree, diploma or other qualifications.

Signed: Jiamin Xu 徐佳敏

Date: April 30, 2025

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Tongqi Wen, whose expert guidance and encouragement were significant in the completion of this dissertation. When I initially selected this research topic, applying LLMs to materials science represented a completely unfamiliar field for me, as my previous work had focused exclusively on integrated circuits. It was Dr. Wen who inspired me to explore this challenging domain and provided continuous support throughout the research process.

I am also particularly grateful to Mr. Siyu Liu for generously sharing his expertise and offering invaluable technical assistance. His patience in repeatedly explaining fundamental concepts played a crucial role in my academic development. This dissertation would not have been possible without their exceptional support.

Thanks to my parents and friends for their accompany during my studies in HKU. I'll forever cherish the memory in this journey.

TABLE OF CONTENTS

ABSTRACT ..... **I**

DECLARATION ..... **II**

ACKNOWLEDGMENTS ..... **III**

TABLE OF CONTENTS ..... **IV**

1 INTRODUCTION **1**

1.1 Research Background and Motivation 1

1.2 Literature Review 1

1.2.1 AI Agent based on Large Language Model 1

1.2.2 AI for Materials Science 2

1.2.3 Retrieval-Augmented Generation 3

1.3 Dissertation Organization 5

2 THEORETICAL BACKGROUND **6**

2.1 Core Principles of Large Language Model 6

2.1.1 Transformer Architecture 6

2.1.2 Softmax function 8

2.2 Large Language Model Based Autonomous Agent 10

2.2.1 Agent architecture 10

2.3 Retrieval-Augmented Generation 12

2.3.1 Basic Retrieval-Augmented Generation 12

2.3.2 Revolution of Retrieval-Augmentation Generation 13

3 INDIVIDUAL LLM PERFORMANCE **15**

3.1 Datasets Construction and LLMs Selection 15

3.1.1 Pymatgen-based Dataset Construction 15

3.1.2 Large Language Models Selection 16

3.2 Performance Evaluation 17

3.2.1 Overall Performance Evaluation 17

3.2.2 Code-based Task vs. Doc-based Task 20

3.2.3 Model-Scale vs. Performance Trade-offs 21

3.3 Error Analysis Error Analysis 21

3.3.1 General Error Patterns 21

3.3.2 Model-Specific Limitations 23

3.4 Future Improvement Directions 23

4 AGENT/RAG ENHANCED LLMS FOR MATERIALS SIMULATION AGENT/RAG ENHANCED LLMS FOR MATERIALS SIMULATION **25**

4.1 Knowledge Base Construction 25
4.1.1 Vector Store Generation 25
4.1.2 LLM Selection 25
4.2 Enhanced RAG Workflow Architecture Enhanced RAG Workflow Architecture 25
4.2.1 Components of RAG 26
4.2.2 Overall workflow 27
4.3 Performance Evaluation 28
4.3.1 Overall Evaluation 28
4.3.2 Qwen2.5-7B-Instruct with/without RAG 28
4.3.3 Darwin1_v2 with/without RAG 29
4.4 Discussion 29
4.4.1 Qwen2.5-7B-Instruct Analysis 30
4.4.2 Darwin1_v2 Analysis 30
4.5 Limitations and Future Improvement Directions 30
4.5.1 Current Limitations 30
4.5.2 Future Improvements 31
5 CONCLUSION 32
REFERENCES 34

1 INTRODUCTION

1.1 Research Background and Motivation

Materials simulation is a multi-scale challenge that demands a comprehensive approach – one that involves retrieving relevant knowledge, applying advanced computational methods, conducting experimental validations, and analyzing the results. Traditionally, this process is very slow and restricted to human experts. Though deep surrogate models can predict material properties from structural features, most current implementations are application-specific, lacking the versatility to handle diverse knowledge domains or unanticipated challenges. To overcome these limitations, AI agent powered by large language models (LLMs) [1][2] and Retrieval-Augmented Generation (RAG) have emerged as a promising solution.

1.2 Literature Review

1.2.1 AI Agent based on Large Language Model

In the past few years, large language models (LLMs) have attracted widespread attention owing to their impressive capabilities across multiple domains [3][4][5], leading researchers to construct AI agents base on LLM [6][7][8]. Specifically, they take LLM as the main component of the agent's brain or central controller modules, and expand their perception and action space through strategies such as multimodal perception and tool usage [9][10]. These LLM-based agents demonstrate significant reasoning and planning capabilities through chain-of-thought and problem decomposition techniques. Unlike reinforcement learning approaches, LLM-based agents offer natural language interfaces for human interaction, enhancing flexibility and explainability. Furthermore, they can learn from feedback and dynamically execute new actions to interact with external environments. Pre-trained on massive corpora, LLMs have demonstrated few-shot and zero-shot generalization capabilities, enabling them to handle diverse tasks without parameter updates [11][12][13].

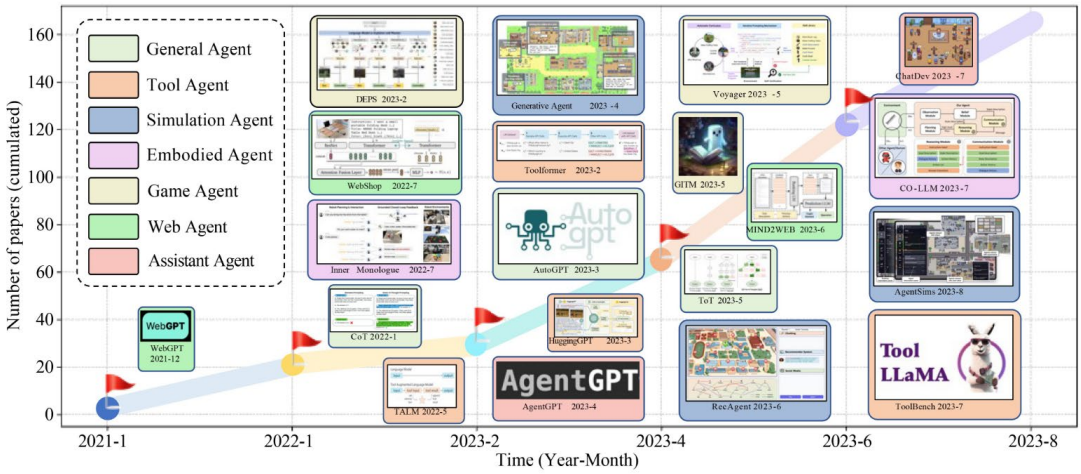

Figure 1.1 Visualization of the expansion trends of LLM-based autonomous agents [14].

As shown in Figure 1.1, researchers have developed numerous promising models along this direction. The core concept involves equipping LLMs with human-like capabilities such as memory and planning, enabling them to perform tasks effectively while imitating human behavior. LLM-based agents have already been deployed in numerous real-world applications, including social science, natural science and engineering. For example, ChatEDA is designed as an EDA (Electronic Design Automation) agent to enhance design processes through integrated task planning, execution and script generation [15].

1.2.2 AI for Materials Science

Artificial Intelligence (AI), including machine learning (ML), deep learning (DL), and large language models (LLMs) has become a essential screening tool for materials scientists across diverse applications, such as energy materials, semiconductors, polymer design, catalysts and high-entropy alloys. A significant trend that shifting from single-dataset processing for specific goals to learning latent representations of underlying structures has emerged. These representations can be fine-tuned for specialized tasks for insurance, predicting energetically stable configurations across multiple datasets [16]. Especially in biomaterials and organic materials [17], notable advances have been made using such methods. This success stems from organic molecules' simple structural representations coupled with the availability of extensive, reliable datasets [18]. However, inorganic materials pose greater complexity for AI applications due to their multiple

properties, particularly when their compositional space includes over four different element types, impeding the discovery and design of new materials in this field.

Current AI applications in materials science predominantly employ conventional machine learning and deep learning approaches, while the utilization of large language models remains an emerging research area requiring further development.

1.2.3 Retrieval-Augmented Generation

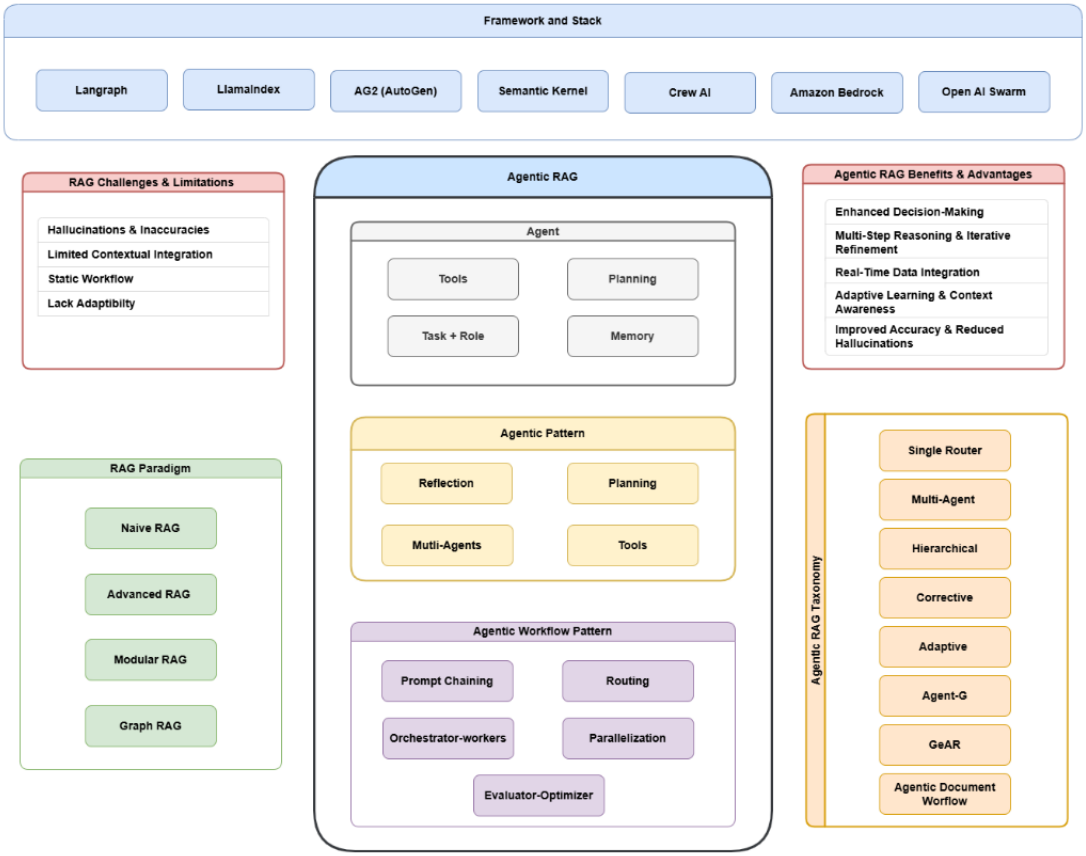

Figure 1.2 An Overview of Agentic RAG [19]

Retrieval-Augmented Generation [20] (RAG) technique improves the reliability and accuracy of generated models by integrating knowledge from external data sources. This technique combines traditional language models with external knowledge bases, and not only enriches the generated content, but also ensures the relevance and timeliness of the content by performing dynamic information retrieval before text generation. RAG systems can be categorized into three types, depending on the stage of enhancement implementation:

enhancement in the pre-training stage, enhancement in the fine-tuning stage, and enhancement in the reasoning stage.

Research on RAG enhanced in the pre-training phase focuses on how to inject external knowledge into the pre-trained model in order to improve the performance of the language model. Guu et al [21] proposed the REALM model which realizes the explicit utilization of the text corpus knowledge of the language model in the pre-training phase by introducing a knowledge retriever. Borgeaud et al [22] designed the RETRO model which effectively reduces the number of participants in the model by retrieving and encoding the document fragments in the corpus, which effectively reduces the number of parameters of the model. Wang et al [23] compared the RETRO model with the Generative Pre-trained Transformer (GPT) [13], which is a converter-based model, and found that RETRO outperforms GPT on knowledge-intensive tasks, and further developed the RETRO++ model to improve the performance of open-domain Q&A tasks.

The fine-tuning stage of enhanced RAGs strives to improve the quality of semantic representations for better adaptation to specific application scenarios. Yu et al [24] designed the augmented adaptive retriever ARR, which enhances the generalization ability of retrieval by learning the language model preferences. Cheng et al [25] proposed a universal cue retrieval model UPRISE, which automatically retrieves relevant cues for zero-sample tasks and improves model multi-task environment. Li et al [26] proposed SANTA, a dense structure-aware retrieval-based model, which optimizes code search performance through structured data alignment and performs well in zero-sample scenarios.

The enhanced RAG in the reasoning stage improves the accuracy of answer generation by integrating richer contextual information. Guo et al [27] proposed the PGRA framework to cope with non-knowledge intensive tasks. The framework first acquires candidate knowledge using a general-purpose retriever, and then ranks the candidates based on task-relevant cues and pre-trained models. Shao et al [28] proposed the ITER-RETGEN approach, which strengthens the collaboration between retrieval and generation through iterative optimization to enhance the relevance of the content and conserve the resource consumption. Trivedi et al [29] proposed the IRCoT framework, which combines the steps

of retrieval and Chain of Thought [30] (CoT), effectively reduces the generation of erroneous results by using Chain of Thought to guide retrieval and optimizing Chain of Thought based on retrieval results.

With the rapid development of large language modeling technology, inference phase-enhanced RAGs have gradually become mainstream and their development architectures have been significantly advanced [31], especially frameworks such as LangChain, LlamaIndex, and AutoGen, which provide a series of modular functions covering data segmentation, import, vectorized storage, retrieval optimization, and answer generation. The integrated functional design greatly simplifies the construction process of the RAG system pipeline and improves the development efficiency and system flexibility. The application of RAG in the field of materials science still needs to be improved in terms of its effectiveness in content generation and data computation due to the limitations of the complexity of the field and diversity of scenarios.

1.3 Dissertation Organization

This dissertation is organized into five chapters:

Chapter 1 establishes the background, research motivation and literature review, framing the research gaps in LLM-based agents for materials science. Chapter 2 presents the theoretical foundations, focusing on principles of LLM, RAG and autonomous agents, as well as their relevance to materials simulation application. Chapter 3 introduces the generation process of Pymatgen-related datasets and evaluates individual LLM performance on both code-based and doc-based tasks. Chapter 4 proposes an LLM-based architecture with RAG enhancement, benchmarking its improvements over baseline models. Finally, Chapter 5 synthesizes key findings, discusses limitations, and outlines future directions for AI in materials simulation.

2 THEORETICAL BACKGROUND

This chapter begins with an introduction to nonlinear functions that are widely used in deep neural networks and explains the features of the Softmax function. Afterwards, it describes improved algorithms that address the performance issues of the Softmax function. Finally, this chapter compares several classical methods for computing nonlinear functions in preparation for subsequent designs.

2.1 Core Principles of Large Language Model

Large language models have achieved great success in various domains during the past few years. In this chapter, we will introduce the core principles of LLMs including the Transformer, which is the model architecture on which LLMs are based, as well as the training and application paradigms of mainstream LLMs.

2.1.1 Transformer Architecture

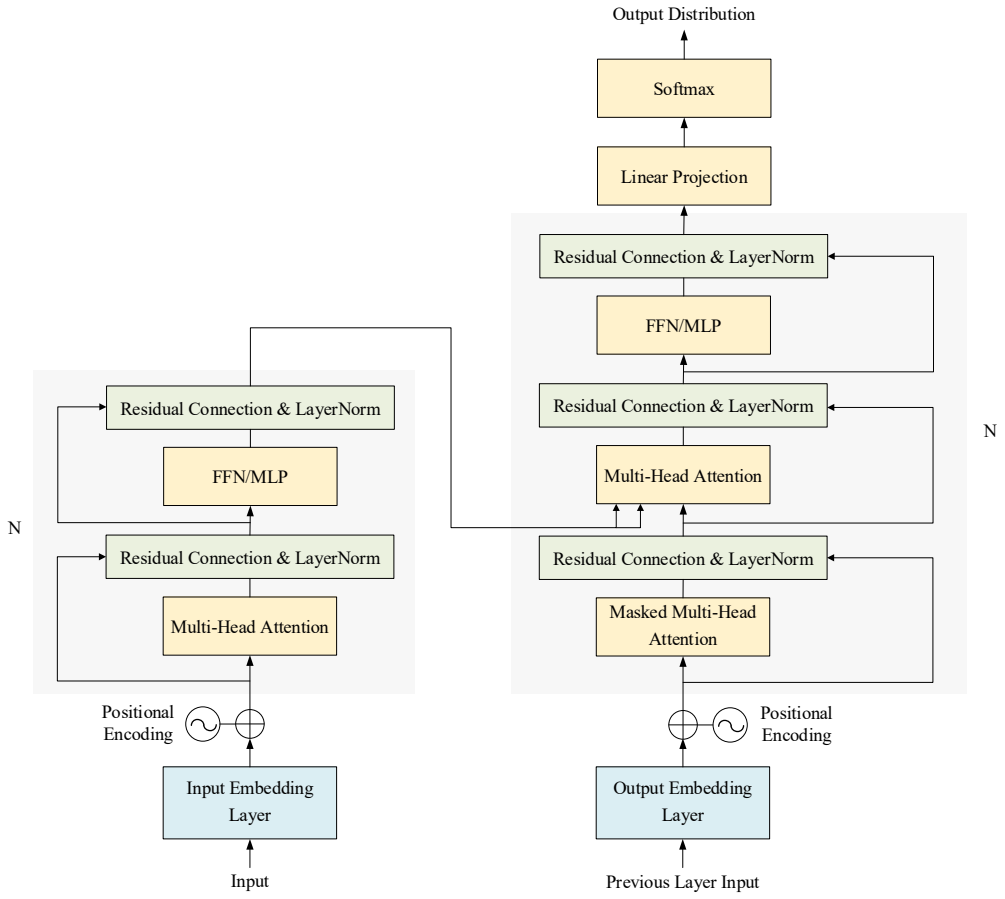

Figure 2.1: Architecture of Transformer-based model

The Transformer is a breakthrough neural network architecture for natural language processing (NLP). Different from tradition recurrent neural networks (RNNs) and convolutional neural networks (CNNs), it uses self-attention mechanism to process sequential data, which improves both efficiency and speed. As shown in Figure 2.1, the core structure of Transformer includes stacker encoder and decoder layers, allowing the Transformer to be capable of deep learning essential for modern LLMs.

Tokenization, positional encoding, attention mechanisms, activation functions, and layer normalization are five essential sections in this structure. They all contributes to the Transformer's advanced language processing and task generation abilities.

Tokenization:

Tokenization splits text into smaller and meaningful units called tokens. This step is essential for LLMs during understanding and generation states. Different tokenization strategies can directly impact on the model's performance.

Character-level tokenization is a strategy that splits text into individual characters. It helps LLMs deal with new words in a simpler and more flexible way yet demands for more computational resources due to loss of word-level information at the same time. Word-level tokenization, just as its name implies, divides text by spaces and punctuation into complete words. This strategy can effectively capture semantic meaning but struggles with vocabulary growth and unknown terms. Subword-level tokenization is the most widely-used tokenization strategy in modern LLMs as it can maintain semantic understanding and meanwhile managing vocabulary size and computational demands efficiently by converting words into meaningful components. Thereby, subword-level tokenization enables efficient performance in the training and inference process of LLMs.

Encoding Positions:

Encoding positions are essential due to their ability to enable models to understand word order in sequences. As we have mentioned, self-attention mechanism lacks the capability of sequential processing. Positions encoding solves this problem by adding positional information to each input token. Common implementations in mainstream LLMs include learnable position

encodings and relative position encodings. Both of these methods offer flexibility for variable-length inputs and can better capture positional relationships.

Attention Mechanism:

By assigning weights to input tokens, attention mechanism can highlight essential information in Transformer-based models. Its working principle is as following: (1) Mapping each token to Query (Q), Key (K), and Value (V) vectors; (2) Calculating attention scores via Q-K interactions; (3) Weighting V with these scores to highlight critical context. Self-Attention, Cross Attention, Sparse Attention and Flash Attention are four Attention strategies which can optimize the performance of LLMs in distinct ways, enabling models to handle complex NLP tasks more effectively.

Activation functions:

Activation function, as the basis for networks to learn complex data patterns as well as fit curves, plays a crucial role in neural networks. Language models commonly use the Rectified Linear Unit (ReLU), Gaussian Error Linear Unit (GeLU), and Gated Linear Unit (GLU) to significantly enhance the neural network's capacity to handle complex tasks by importing non-linear transformations and gating mechanisms.

Layer Normalization:

Layer normalization (LayerNorm) is a fundamental technique widely used in neural networks, particularly in sequence-processing models like Transformers. The core principle is to normalize input features within each network layer to standardize feature scales. This normalization helps control data scale in network propagation, effectively addressing common training challenges such as gradient vanishing or explosion.

2.1.2 Softmax function

This section presents the fundamental techniques of LLMs, including pre-training, fine-tuning and methods applied for downstream tasks. Figure 2.2 illustrates the workflow of the training states and inference process in basic LLMs.

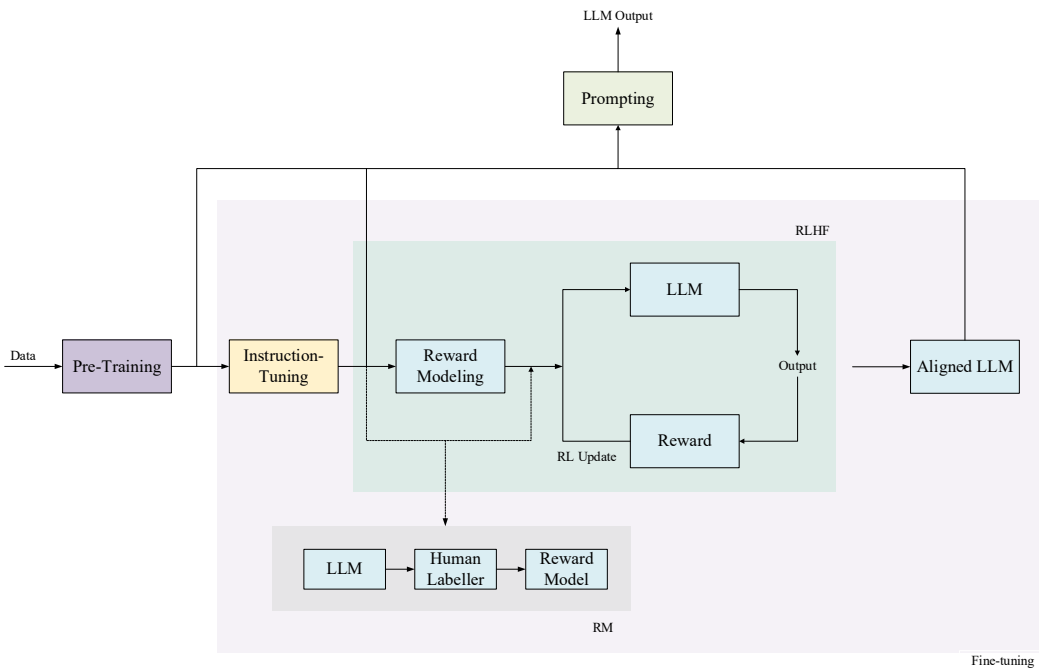

Figure 2.2: The basic workflow of LLMs

Pre-training

In the state of pre-training, LLMs acquire fundamental linguistic structures and semantics by learning general language representations from extensive textual datasets. The selection of pre-training strategies will profoundly affect the model's performance and applicability.

Self-supervised learning is a pre-training strategy where a model learns the intrinsic structure of the data by predicting missing parts or future sequences in the input data. For LLMs, this usually means predicting the next word or masked word based on the given textual context (e.g., the masked language modeling task in BERT). The advantage is that there is no need for labeled data, a large amount of unlabeled text data can be utilized, and this approach helps to learn deep linguistic representations and improve the model's understanding of linguistic structure. But again, there may be a gap between this pre-training approach and the downstream task that needs to be bridged by fine-tuning.

Fine-Tuning

The fine-tuning state is the process of adapting a generic pre-trained model to a specific task. Some of the current mainstream methods are as following:

(1) Transfer Learning: Pretrained models have demonstrated strong performance across various tasks. However, to further enhance their effectiveness on specific downstream tasks, researchers typically fine-tune these pretrained models on task-specific datasets, a process known as transfer learning.

(2) Instruction-tuning: To make the model more effective in response to user queries, researchers fine-tune the pre-trained model on formatted datasets containing instructions and input-output pairs. This fine-tuning approach can significantly improve the model's performance on unseen tasks and downstream tasks.

(3) Alignment-tuning: LLMs are likely to generate inaccurate, biased, or harmful text sometimes, so aligning the model through human feedback becomes particularly important. This process guides the model to generate unintended responses and updates model parameters to avoid unexpected outputs. The ultimate goal is to ensure that LLMs are able to function according to human intentions and values.

Prompt Engineering

As illustrated in Figure 2.2, prompt engineering is a method to generate responses by querying trained LLMs. This process can be implemented with various configurations. In some certain scenarios, it may not require additional fine-tuning, while others might need fine-tuning with datasets incorporating diverse questioning styles.

2.2 Large Language Model Based Autonomous Agent

Autonomous agents are widely considered as a crucial way to develop artificial general intelligence (AGI) because it has demonstrated remarkable capabilities in autonomous task execution through self-guided planning and decision-making processes. This section provides an overview of LLM-based autonomous agents focus on their prevailing architecture and diverse applications across various domains.

2.2.1 Agent architecture

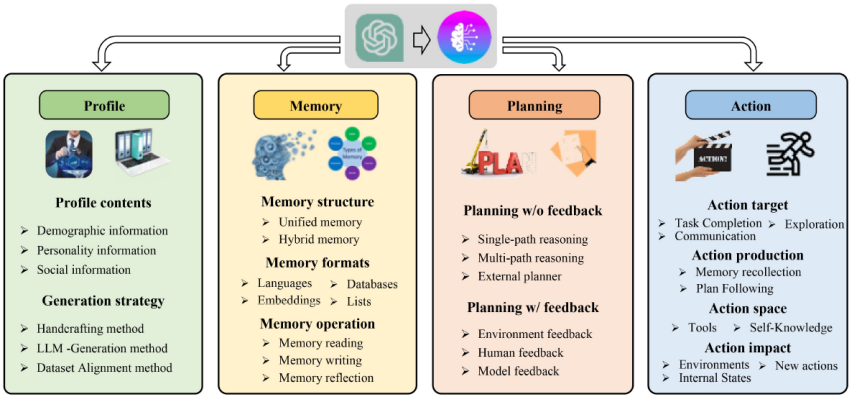

Figure 2.3: A unified framework for the architecture design of LLM-based autonomous agent [31]

To enhance the functionality of basic LLMs and combined them with autonomous agents, the key challenge lies in how to design effective architecture that optimize their inherent ability. Figure 3 shows an advanced framework of autonomous agents that incorporates four core components: Profiling module, Memory module, Planning module, and Action module. The functions and interactions of these components will be discussed in the followings.

Profiling module

Autonomous agents perform specialized tasks by taking on well-defined roles, for example in this research, a materials scientist. The profiling component is essential in shaping agent behavior by adding key characteristics into the LLM's prompts. These characteristics typically consist of basic biographical information, psychological traits to define the agent's personality, and social context to determine its relationships with other agents. You can choose the specific composition depend on what missions you want the agent to complete. Current research raises three primary methods to create the profiles: manual specification, alignment with real datasets, and automated generation by LLMs. The profiling system forms the foundation of agent, significantly affect how agents will store information, make decisions, and execute actions.

Memory module

Memory module is considered as an essential bridge connecting perception state and planning state. To be specific, it stores information perceived from the outer environment and leverages the recorded memories to facilitate future actions. The agent capability of accumulating experiences, self-evolving mainly relies on this module, which leads the agent to behave in a more reasonable, effective and consistent manner. Agent's memory is usually distinguished short-term memory to long-term memory modeling human memory. Short-term memory temporarily buffers recent perceptions, including the context information about the agent current situations, while long-term memory consolidates essential information such as past behaviors and thoughts over time.

Planning module

Planning module enables the agents to deconstruct a complex task and solve them individually, making the agent behave more reliably, powerfully and humanlike. Single-path reasoning with

the agent follows one straight path to solve the problem and multi-path reasoning with the agent considers multiple possible solutions are two strategies in planning without feedback approach. The implementation of this approach is relatively simple but it is only suitable for simple tasks requiring few reasoning steps. As for planning with feedback approach, agents can learn from their mistakes and improve by considering environment feedback, human feedback and model feedback. Although better designs are demanded to handle the feedback, it is regarded as an effective approach to address complicated tasks with long-range reasoning.

Action module

As the final component in the agent architecture, the actions module directly interacts with external systems under the guide of the profile, memory and planning modules. The basic function of this module is to execute the agent's decisions in real-world environments. Action goals (task completion, communication and environment exploration), action generation (memory-based and plan-based), action types (external tools and internal LLM capabilities) and action outcomes (environment modification, internal state updates and action chaining) are four key components of this module, needed to be carefully considered.

2.3 Retrieval-Augmented Generation

2.3.1 Basic Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) are developed to address LLMs limitation in responding to dynamic and real-time queries due to their reliance on static training datasets. By combining the generative capabilities of LLMs to with external retrieval mechanisms, RAG can significantly improve response quality as they can access up-to date information from diverse sources including knowledge repositories, web APIs and online databases.

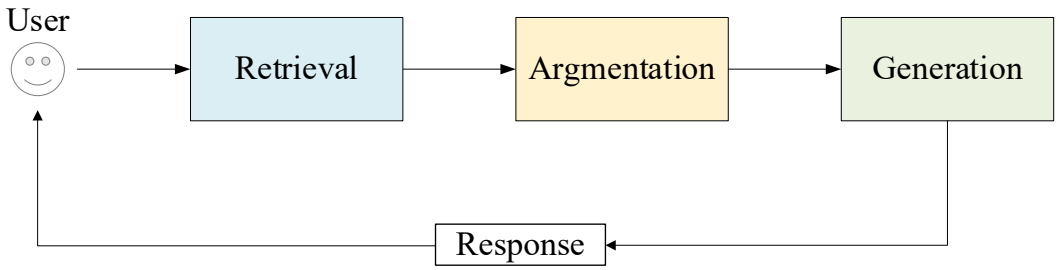

Figure 2.4：Core Components of RAG

Retrieval-Augmentation Generation, as the name suggests, is consisted of three core components shown in Figure 2.4. The further explanation and function of each component are as below:

(1) Retrieval access external data repositories including knowledge bases, APIs and vector databases dynamically to ensure query-relevant results. By using dense vector search combined with models based on Transformer, advanced retrievers are regarded as efficient solution to enhance retrieval precision and sematic relevance.

(2) Augmentation is responsible for processing retrieved data, extracting key information while maintain semantic coherence with the input query.

Generation emerges retrieved context with LLM's existed database to produce responses that balance factual accuracy with linguistic fluency.

2.3.2 Revolution of Retrieval-Augmentation Generation

Over the past few years, the rapid development of RAG technology has necessitated its evolution to address increasingly complex real-world challenges efficiently and effectively. This progression has driven significant improvements in three critical dimensions: contextual precision, system scalability, and multi-stage reasoning capabilities. Next, we will have discussion on several main stages of the RAG progression, comparing their defining characteristics, advantages and limitations.

Naïve RAG

The key aspect of Naïve RAG lies in implementing simple keyword-based retrieval and extract documents from static datasets. The retrieved contents are used in augmentation of LLM generative capabilities. While this kind of RAG is suitable for simple fact-based queries task due to its straightforward structure, it doesn't perform well when processing large datasets. Its lack of contextual awareness often results in fragmented outputs and failure in identifying the key relevant information.

Advanced RAG

Evolving from Naïve RAG's framework, Advanced RAG significantly enhances retrieval precision and contextual relevance through the integration of semantic understanding with optimized retrieval methodologies. It uses dense vector search, contextual re-ranking and iterative retrieval techniques to achieve higher precision and fine-grained understanding in

complex applications. However, this method still struggles to perfectly handle large datasets or multi-step queries.

Modular RAG

As the latest version of RAG systems, Modular RAG is widely used in complicated tasks due to its superior precision and scalable performance across diverse application domains. Hybrid retrieval, tool integration with APIs and composable pipelines are three fundamental innovations in the Modular RAG system, contributing to domain-specific optimization and task adaptability. The system's flexibility and customization come from its modular design, which separates retrieval and generation into independent, reusable parts that can be optimized individually.

Agentic RAG

Agentic RAG introduces a groundbreaking approach by combining autonomous agents with RAG systems, enabling dynamic decision-making and intelligent workflow management. Different from tradition system with static retrieval methods, Agentic RAG implements adaptive strategies and continuous refinement processes to handle complex, multi-domain queries in real-time. However, these advanced capabilities come with increased computational demands and system coordination challenges, particularly when scaling to handle high-volume query environments. The technology represents a significant step forward in making RAG systems more responsive and contextually-aware, though careful resource management remains essential for optimal deployment.

3 INDIVIDUAL LLM PERFORMANCE

3.1 Datasets Construction and LLMs Selection

3.1.1 Pymatgen-based Dataset Construction

We utilize Gemini-2.0-flash to generate two question-answering datasets focused on Pymatgen:

1. A code-based dataset, assessing comprehension of implementation details in materials simulation scripts.
2. A documentation-based (doc-based) dataset, evaluating knowledge of API usage and theoretical concepts.

For the code-based dataset, we employ the large language model to automatically generate five multiple-choice questions per Python code snippet. Each question presents the code with a strategically masked component, denoted as [MASK], where the obscured element corresponds to a key function name or critical code segment essential to the program's logic. The answer choices include both the correct option and carefully designed distractors, which consist of plausible but incorrect alternatives to rigorously assess the reader's understanding. This dataset is designed to evaluate proficiency in code implementation details, encompassing function calls, critical logical constructs, and syntactic conventions. To answer correctly, readers must infer the masked component by analyzing the surrounding context, including function signatures, parameter usage, and operational semantics. Code-based answers require dynamic interpretation of code behavior and dependencies.

The structure of code-based QAs
<question1>Generated question text here with a [MASK] token replacing a function or key part.</question1>
<answer_choices1>
<choice>A. First answer choice</choice>
<choice>B. Second answer choice</choice>
<choice>C. Third answer choice</choice>
<choice>D. Fourth answer choice</choice>
</answer_choices1>
<correct_answer1>Correct choice letter</correct_answer1>

For the doc-based dataset, we employ the same model to generate five multiple-choice questions per Pymatgen documentation segment. This evaluation framework assesses three key

dimensions of documentation comprehension: module architecture understanding, materials science principles, and technical specification interpretation. Readers must demonstrate the ability to synthesize contextual documentation details, integrate cross-domain conceptual knowledge, and strictly adhere to official technical specifications. The distractor design incorporates common misconceptions, conceptually similar but incorrect alternatives, partially accurate options, and documentation-inconsistent variations, ensuring robust evaluation of true understanding. Doc-based answers are strictly verifiable against canonical documentation.

The structure of doc-based QAs
<question1>Generated question text here.</question1>
<answer_choices1>
<choice>A. First answer choice</choice>
<choice>B. Second answer choice</choice>
<choice>C. Third answer choice</choice>
<choice>D. Fourth answer choice</choice>
</answer_choices1>
<correct_answer1>Correct choice letter</correct_answer1>

The code-based dataset contains 34621 QAs while the doc-based dataset contains 34604 QAs. We expect that all question-answering pairs in two datasets will cost a total of 27505032 tokens. These two datasets will serve as benchmarks to investigate the inherent capabilities of various large language models (LLMs) in materials science tasks without retrieval-augmented generation (RAG), thereby isolating their core understanding and reasoning performance.

3.1.2 Large Language Models Selection

The selection of models for our experimental work was guided by the imperative to assess capabilities in materials simulation across a broad spectrum of large language models known for their robust performance in diverse applications. We chose models from the Gemini series (Gemini-1.5-flash, Gemini-1.5-pro, Gemini-2.0-flash, Gemini-2.0-pro-exp-02-05) and the Qwen series (including Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, Qwen-2.5-Coder-14B-Instruct, Qwen2.5-32B-Instruct, Qwen-2.5-Coder-32B-Instruct, Qwen2.5-72B-Instruct-GPTQ-int4, and QwQ-32B) due to their widespread use and proven efficacy in complex language tasks. These models span the gamut from lightweight, with parameter sizes of 7 billion,

to very large, with parameter sizes of 72 billion, representing the cutting edge in generalized language understanding. They provide a robust baseline for benchmarking.

In addition, we incorporated ChenDFM-v1.5-8B, a specialized model with 8 billion parameters, tailored specifically for molecular design, material property prediction, and even chemistry quizzes. We also included Darwin1_v2, which is focused on material simulation, structural optimization, and performance prediction. The strategic inclusion of both general-purpose and specialized models allows us to illustrate how various models perform differently on the two Pymatgen-based datasets we generated. This comparative analysis is crucial for determining the optimal underlying large language model to be integrated with Retrieval-Augmented Generation (RAG) in the subsequent chapter.

3.2 Performance Evaluation

3.2.1 Overall Performance Evaluation

We evaluated the performance of diverse models on code-based dataset and doc-based dataset, including Qwen series models, Gemini series models and two domain-specialized models (ChemDFM-v1.5-8B and Darwin1_v2). The evaluation results are illustrated in Table 1, Table 2 and Table 3.

The benchmark evaluation of Qwen series models (Table 3.1) demonstrates clear performance trends across different model sizes and architectures. Standard Qwen models show progressively higher accuracy with increasing parameter counts, rising from 74.37% for code-based tasks with the 7B model to 81.36% with the 72B models. The performance on doc-based dataset consistently achieves 1-4 percentage points higher accuracy than corresponding the code-based dataset across all model sizes. The coder-optimized models exhibit specialized advantages, with Qwen2.5-Coder-14B performing nearly the same as its standard model on code-based tasks. Notably, the 32B standard model delivers the most balanced performance (80.03% code-based and 82.01% doc-based), while QwQ-32B shows exceptional code-based task capability at 84.27%, which is the highest in the series, though doc-based dataset data remains unavailable due to time limitation. The results suggest that while model scale generally

improves performance, the benefits diminish beyond 32B parameters, and specialized models can achieve targeted improvements over their general-purpose models.

Index	Model	Dataset	Accuracy
1	Qwen2.5-7B-Instruct	Code	74.37
2		Doc	75.79
3	Qwen2.5-14B-Instruct	Code	76.72
4		Doc	80.26
5	Qwen2.5-Coder-14B-Instruct	Code	77.44
6		Doc	79.44
7	Qwen2.5-32B-Instruct	Code	80.03
8		Doc	82.01
9	Qwen2.5-Coder-32B-Instruct	Code	79.18
10		Doc	79.85
11	Qwen2.5-72B-Instruct-GPTQ-int4	Code	81.36
12		Doc	81.82
13	QwQ-32B	Code	84.27
14		Doc	-

Table 3.1: Basic evaluation with various LLMs from the Qwen series across parameter scales (7 billion to 72 billion) reveals that the average accuracy rates of Qwen series models on both benchmarks stabilize at approximately 80%, with modestly higher accuracy observed on doc-based tasks compared to code-based tasks.

Our comparative benchmark analysis of Gemini series models (Table 3.2) reveals several noteworthy performance patterns. The Gemini-1.5-Pro model achieves the highest overall accuracy among all tested models, with 80.60% on code-based datasets and 82.90% on doc-based datasets, demonstrating its strong capability in materials science applications. While the two 1.5 generation models show stable performance across both datasets, the Gemini-2.0-Flash exhibits a significant discrepancy between code (73.86%) and documentation (82.76%) tasks, suggesting potential optimization differences in its architecture. The experimental Gemini-2.0-Pro-Exp-02-05 model shows promising results on code tasks (79.69%), though data of doc-based dataset is currently unavailable due to some reasons would be discussed later. Across all models, doc-based tasks generally yield higher accuracy than their code-based counterparts, with performance gaps ranging from 0.78% (Gemini-1.5-Flash) to as much as 8.9% (Gemini-2.0-Flash), indicating that these models may be more adept at conceptual understanding than code implementation tasks in the materials science domain.

Index	Model	Dataset	Accuracy
1	Gemini-1.5-Flash	Code	75.59
2		Doc	76.37
3	Gemini-1.5-Pro	Code	80.60
4		Doc	82.90
5	Gemini-2.0-Flash	Code	73.86
6		Doc	82.76
7	Gemini-2.0-Pro-Exp-02-05	Code	79.69
8		Doc	-

Table 3.2: Preliminary evaluation of multiple Gemini series models (including Gemini-1.5 and Gemini-2.0) demonstrates an average accuracy of 79% on both tasks, with doc-based evaluations consistently outperforming code-based tasks.

The benchmark evaluation of domain-specific models (Table 3.3) reveals significant performance limitations compared to general-purpose LLMs. ChemDFM-v1.5-8B demonstrates measurable but substantially lower accuracy (32.35% on code-based dataset and 30.20% on doc-based dataset), while Darwin1_v2 shows near-zero task comprehension with only 0.01% accuracy across both datasets. This result pronounces performance gap of over 30 percentage points between the two specialized models and more than 50 points compared to Qwen and Gemini's average accuracy (~80%), suggesting fundamental challenges in applying these domain-specific LLMs to Pymatgen-related tasks. Notably, both models exhibit minimal variation between code and documentation task performance (differences <2.5%), contrasting with the significant benchmark disparities observed in general-purpose models. The particularly poor performance of Darwin1_v2, despite its material science specialization, indicates potential limitations in either its training methodology or fundamental architecture for these material simulation tasks. These results highlight the need for careful validation when implementing domain-specific models and the great potential in these models to improve their performance in materials simulation.

Index	Model	Dataset	Accuracy
1	ChemDFM-v1.5-8B	Code	32.35
2		Doc	30.20
3	Darwin1_v2	Code	0.01
4		Doc	0.01

Table 3.3: Comparative evaluation with the two domain-specialized models. The result reveals stark performance differences: ChemDFM-v1.5-8B achieved modest accuracy (30.20-32.35%), while Darwin1_v2 failed to demonstrate meaningful task comprehension (0.01% accuracy across both datasets).

3.2.2 Code-based Task vs. Doc-based Task

In the dataset generation chapter, we highlight the difference of prompts designed for generating the code-based dataset and the doc-based dataset. Therefore, the performance of LLMs differs on the experiments on the two Pymatgen-related datasets. The code-based dataset usually demands the LLMs to interpret dynamic code context, challenging the capability of syntax awareness and control flow analysis. As for the doc-based dataset, the difficulty mainly focuses on specialization of materials science terminology and precise recall of documented specifications.

As shown in Figure 3.1, doc-based tasks achieve significantly higher accuracy rates than code-based tasks. Aligned with Bloom's Taxonomy, code-based tasks assess higher-order skills (Levels 4-6: Analyze/Evaluate/Create), while doc-based questions target foundational knowledge (Levels 1-3: Remember/Understand/Apply). This explains the observed accuracy gap in LLMs performance.

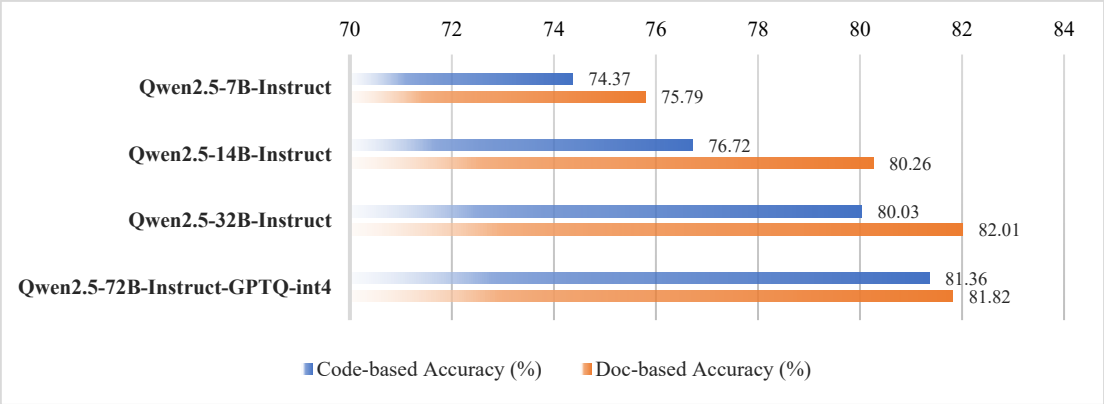

Figure 3.1: Comparison on code-based accuracy and doc-based accuracy across Qwen2.5-Instruct models ranging from 7 billion to 72 billion parameters. The result reveals that the model accuracy shows consistent improvement with increasing parameter counts across both dataset types. Also, tests on doc-based datasets achieve higher accuracy over code-based datasets.

3.2.3 Model-Scale vs. Performance Trade-offs

Figures 3.1 and 3.2 illustrate two key observations regarding the Qwen2.5 model series (7B to 72B parameters): 1. Accuracy-Parameter Relationship: A clear positive correlation exists between model size and benchmark accuracy, with the 72B parameter model achieving the highest performance (81.36% code accuracy, 81.82% documentation accuracy). 2. Computational Cost: This accuracy improvement comes with significantly increased inference time, revealing a distinct performance-efficiency trade-off. The complete 72B model requires substantially more computational resources than its smaller counterparts.

Notably, the Qwen2.5-72B-Instruct-GPTQ-int4 variant (a 4-bit quantized version of the 72B model through GPTQ compression) demonstrates deviation from this trend. Its accuracy and runtime characteristics reflect the unique constraints of quantization, rather than following the standard parameter scaling pattern observed in full-precision models.

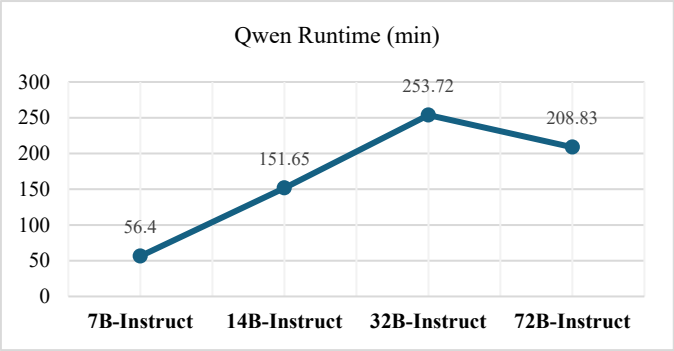

Figure 3.2: Comparison on runtime cost across Qwen2.5 models ranging from 7B to 72B parameters on code-based dataset.

3.3 Error Analysis Error Analysis

This section analyzes the error patterns observed during evaluation, including both systemic limitations and model-specific failure modes.

3.3.1 General Error Patterns

Answer Matching Inconsistencies

The prompt used to query LLM responses for the Pymatgen datasets:
1: question_test_prompt = (

```
    "You are a materials scientist with expertise in Pymatgen for solving material simulation problems. "
    "Below is a multiple-choice question related to Pymatgen. Please select the correct answer based on your expertise.\n\n"
    "<question>{question}</question>\n"
    "<answer_choices>\n"
    "    <choice>{first_choice}</choice>\n"
    "    <choice>{second_choice}</choice>\n"
    "    <choice>{third_choice}</choice>\n"
    "    <choice>{fourth_choice}</choice>\n"
    "</answer_choices>\n\n"
    "Only provide your answer as a single letter (A, B, C, or D), formatted in tags as follows:\n"
    "<answer>Your answer here</answer>"
)

extracted_answer =
llm_answer.strip().replace("<answer>", "").replace("</answer>", "").strip()
```

While our prompt strictly requires models to output only a single letter (A/B/C/D) between <answer> and </answer> tags, the observed model behaviors demonstrate notable deviations from this instruction. Specifically, we identify two recurring patterns of non-compliance: LLMs frequently output both the option letter and its corresponding content between the answer tags (e.g., <answer>B. lattice_vectors</answer>), rather than providing just the letter as specified. Additionally, models employing chain-of-thought reasoning often insert their reasoning processes within the answer tags (e.g., <answer>Based on the crystal structure analysis, the correct answer is C</answer>), despite explicit instructions to the contrary.

These systematic deviations render our current answer extraction methodology, which simply preserves all content between the answer tags, inadequate for reliable evaluation. To address these extraction challenges, we will introduce a specialized parsing agent designed to precisely extract LLM responses using an optimized answer extraction methodology in the following chapter.

Template Misalignment

Domain-specialized models like Darwin and ChemDFM-v1.5-8B exhibit significant limitations when processing open-ended prompts designed for general-purpose LLMs, primarily due to their constrained training paradigms. These models, typically fine-tuned on narrow datasets with fixed Q&A formats, often fail to properly interpret more flexible prompt structures. A common failure mode involves the models either literally outputting template placeholders like "your answer here" as their response, or rigidly adhering to their training format by prefixing answers with unnecessary labels (e.g., automatically adding "The answer is: " before responses). This behavior stems from their encoder-focused architectures and limited exposure to diverse instruction types during training. The issue is particularly pronounced in this experiment where precise formatting and technical accuracy are crucial.

3.3.2 Model-Specific Limitations

Our evaluation reveals distinct constraints across model categories. General-purpose LLMs (Gemini/Qwen series) demonstrate fundamental domain knowledge gaps due to lacking materials science-specific fine-tuning. Otherwise, these models' performance is further obscured by current exact-match method that fail to capture semantically equivalent but syntactically varied responses.

Conversely, domain-specialized models (Darwin1_v2 and ChemDFM-v1.5-8B) exhibit opposite limitations: their rigid architectures, optimized for fixed-format queries in narrow domains, struggle with the flexible prompt structures required for general benchmarking. This specialization creates brittle input parsing behaviors and poor generalization beyond their predefined task scopes, particularly when handling open-ended materials science problems. The dichotomy highlights an intrinsic tension between breadth and depth in LLM specialization for scientific domains.

3.4 Future Improvement Directions

To address the identified limitations and improve the reliability of LLMs in materials simulation, we propose two key enhancements to the standard workflow:

1. Retrieval-Augmented Generation (RAG) Integration:

By incorporating specific knowledge about materials science repositories, the RAG component is designed to mitigate domain-specific knowledge gaps prevalent in general-purpose LLMs. This augmentation is particularly targeted at reducing factual inaccuracies in API usage and theoretical concepts.

2. LLM-Based Answer Extraction Agent:
   A specialized extraction model will process raw LLM outputs to align responses with evaluation standards, addressing current template mismatching problems. This agent performs critical normalization of technical notations and structural reformatting of answers.

The implementation details and empirical validation of these enhancements will be systematically presented in the following chapter, including quantitative comparisons of pre and post optimization performance metrics.

4 AGENT/RAG ENHANCED LLMS FOR MATERIALS SIMULATION AGENT/RAG ENHANCED LLMS FOR MATERIALS SIMULATION

4.1 Knowledge Base Construction

In the previous chapter, we generated two Pymatgen-based question-answering datasets using the Gemini-2.0-flash model. To enhance the performance of large language models (LLMs) in materials simulation through Retrieve-Augmented Generation (RAG), the original JSON source datasets must be converted into vector stores. This transformation enables the RAG system to effectively process and understand the materials science data for improved retrieval and generation process. In this chapter, we select two language models - Qwen2.5-7B-Instruct and Darwin1_v2 - from our previously evaluated LLM candidates (Chapter 4) to assess their performance on the same datasets using RAG and a format-checking agent.

4.1.1 Vector Store Generation

4.1.2 LLM Selection

4.2 Enhanced RAG Workflow Architecture Enhanced RAG Workflow Architecture

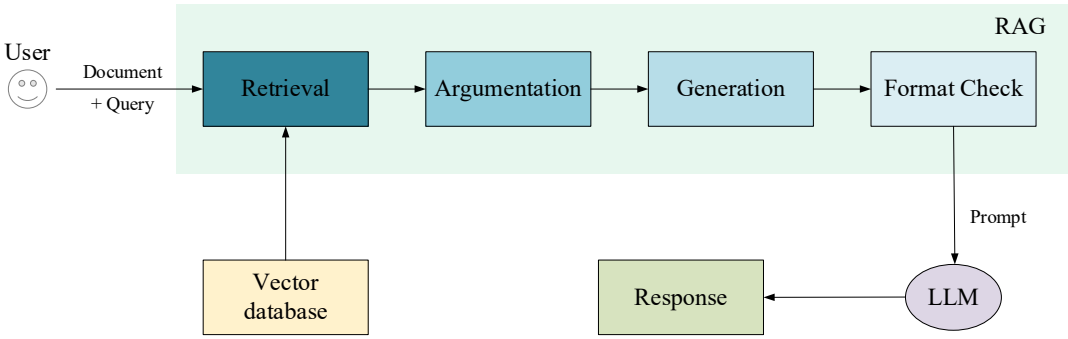

Figure 4.1: Overview of the enhanced RAG workflow architecture

The enhanced workflow architecture incorporating RAG with format verification function is shown in Figure 4.1. This system extends the previous pipeline by first initializing two local

LLMs (a primary generation model and a dedicated format verification model) along with the pre-generated vector database. The core Retrieval-Augmented Generation process then executes, handling queries through sequential stages of context retrieval, response generation, and format validation. Finally, the evaluation module calculates accuracy rates using standardized assessment protocols while maintaining detailed execution logs for comprehensive performance analysis. This integrated architecture facilitates evaluation of RAG augmentation effects while ensuring output quality through systematic format verification.

4.2.1 Components of RAG

Figure 5.1 presents the RAG architecture in this paper, implemented using LangGraph to create a stateful multi-stage processing pipeline. The system comprises three core operational states including retrieval state, generation state and format-checker state. Each state maintains its own execution context while participating in the coordinated workflow. The following section will provide detailed explanation of each component's implementation and functions.

Workflow based on LangGraph

Workflow based on LangGraph

```
def build_graph(self):
    workflow = StateGraph(State)
    workflow.add_node("retrieve", self.retrieve)
    workflow.add_node("generate", self.generate)
    workflow.add_node("format_checker", self.format_checker)

    workflow.add_edge(START, "retrieve")
    workflow.add_edge("retrieve", "generate")
    workflow.add_edge("generate", "format_checker")
    workflow.add_edge("format_checker", END)
```

Retrieval State

The Retrieval State queries the vector database with semantic similarity matching to extract the most relevant document fragments. This process identifies top-5 contextual passages that maximally align with the query's semantic content. Raw text content (page_content) of each retrieved document is extracted and aggregated into a single string, with individual contexts

separated by distinctive demarcation lines to maintain clear boundaries between different source materials. This state returns these compiled contexts packaged in a dictionary format (context), providing properly formatted reference materials for subsequent Generation State.

Generation State

The Generation State synthesizes answers by processing retrieved knowledge and input questions through the main model while enforcing strict formatting protocols. With specific prompt, the system configures the model as a materials science specialist with expertise in Pymatgen, constraining output answers to single-letter selections (A-D). This controlled generation process transforms the formatted question and retrieved content into standardized responses encapsulated within <answer></answer> tags, guaranteeing both technical accuracy in materials simulation and structural consistency for downstream pipeline operations.

Format-checker State

The Format-checker State enforces output standardization by transforming raw model-generated responses into strictly compliant single-letter answers. The system provides a specialized prompt to a format-check model and ask it to extract only the essential answer content by systematically eliminating all on-conforming elements. This validation mechanism is expected to guarantee output consistency by converting potentially verbose or irregular generation outputs into atomic answer units, effectively decoupling content accuracy from presentation variability to ensure reliable downstream processing.

4.2.2 Overall workflow

The core architecture maintains consistency with our previously designed framework in Chapter 4, while incorporating the new RAG-enhanced components. The system follows a standardized workflow: it initially generates properly formatted questions from the Pymatgen-based datasets, then processes these questions through the RAG pipeline to obtain structured answers. Following completion, the system automatically calculates overall accuracy and generates two types of output files for performance analysis - a detailed CSV report containing quantitative evaluation results and a timestamped log file recording all processing details. This enhanced version preserves the original architecture’s evaluation methodology while benefiting

from the improved answer quality enabled by the RAG augmentation, allowing for direct comparison between baseline and enhanced model performance. The output files maintain consistent formatting with our previous implementation to ensure continuity in experimental tracking and result analysis across different phases of our research.

4.3 Performance Evaluation

Our evaluation assessed the RAG-enhanced performance of two models (Qwen2.5-7B-Instruct and Darwin1_v2) with integrated format checker on both code-based and documentation-based datasets.

4.3.1 Overall Evaluation

Table 4.1 compares the RAG-enhanced performance of Qwen2.5-7B-Instruct and Darwin1_v2 models across code-based and document-based material science datasets. The results reveal significant performance disparities: Qwen2.5-7B-Instruct achieves 77.10% accuracy on code tasks and 82.92% on documentation tasks, demonstrating a consistent performance advantage in text processing scenarios. In stark contrast, the domain-specific Darwin1_v2 model shows negligible effectiveness (0.99% accuracy on both datasets), suggesting fundamental incompatibility with these material science tasks.

Index	Model	Dataset	Accuracy (%)
1	Qwen2.5-7B-Instruct	Code	77.10
2		Doc	82.92
3	Darwin1_v2	Code	0.99+
4		Doc	0.99+

Table 4.1: Evaluation on RAG-enhanced LLMs on code-based and doc-based datasets.

4.3.2 Qwen2.5-7B-Instruct with/without RAG

Table 4.2 presents a comparative analysis of Qwen2.5-7B-Instruct's performance with and without RAG enhancement across code-based and doc-based material simulation tasks. The results demonstrate consistent accuracy improvements through RAG augmentation, with a 2.73 percentage point increase on code-based tasks (from 74.37% to 77.10%) and a more substantial 7.13 percentage point gain on doc-based tasks (from 75.79% to 82.92%). Notably, the greater performance boost observed in documentation processing (7.13% vs 2.73% for code) suggests

RAG's particular effectiveness in enhancing text-based material simulation applications. The consistent positive differentials across both dataset types confirm the value of retrieval augmentation for this general-purpose model in materials simulation applications.

Feature	Dataset	Standard	RAG	Differences
Accuracy	Code	74.37	77.10	+2.73
(%)	Doc	75.79	82.92	+7.13

Table 4.2: Comparison of Qwen2.5-7B-Instruct with/without RAG enhancement performance.

4.3.3 Darwin1_v2 with/without RAG

Table 4.3 reveals fundamental limitations in Darwin1_v2's performance across material application tasks, showing near-zero baseline accuracy (0.01%) that improves only marginally to 0.99% with RAG enhancement. The identical nearly 1.00 percentage point improvement for both code-based and doc-based tasks suggests systemic constraints in the model's architecture or configuration rather than dataset-specific deficiencies. This consistent pattern across experimental conditions implies that neither standard implementation nor RAG augmentation can overcome Darwin1_v2's inherent incompatibility with these material simulation applications. The results strongly suggest the need for architectural reconsideration or configuration modification rather than supplementary techniques to achieve viable performance.

Feature	Dataset	Standard	RAG	Differences
Accuracy	Code	0.01	0.99	+0.98
(%)	Doc	0.01	0.99	+0.98

Table 4.3: Comparison of Darwin1_v2 with/without RAG enhancement performance.

4.4 Discussion

Three primary categories of errors emerged from our previous assessment in chapter 4: (1) content generation of errors emerged from domain knowledge gaps; (2) answer format violations; (3) model-specific abnormal behaviors. The Qwen2.5-7B-Instruct predominantly exhibited type (1) and (2) errors, reflecting its general-purpose architecture's limitations in specialized materials science applications. In contrast, Darwin1_v2 demonstrated near-total type (3) failures, suggesting fundamental incompatibility with the task requirements.

With the enhance of RAG, Qwen2.5-7B-Instruct showed meaningful improvements with 7.13% increase in accuracy on doc-based datasets. However, RAG has a marginal impact on Darwin1_v2, which suggests deeper architectural or training flaws of this model.

4.4.1 Qwen2.5-7B-Instruct Analysis

Three primary categories of errors emerged from our previous assessment in chapter 4: (1) content generation of errors emerged from domain knowledge gaps; (2) answer format violations; (3) model-specific abnormal behaviors. The Qwen2.5-7B-Instruct predominantly exhibited type (1) and (2) errors, reflecting its general-purpose architecture's limitations in specialized materials science applications. In contrast, Darwin1_v2 demonstrated near-total type (3) failures, suggesting fundamental incompatibility with the task requirements.

With the enhance of RAG, Qwen2.5-7B-Instruct showed meaningful improvements with 7.13% increase in accuracy on doc-based datasets. However, RAG has a marginal impact on Darwin1_v2, which suggests deeper architectural or training flaws of this model.

4.4.2 Darwin1_v2 Analysis

The demonstrated accuracy improvements in Qwen2.5-7B-Instruct strongly suggest that RAG augmentation can effectively enhance LLM performance in materials science applications. In contrast, Darwin1_v2's persistent poor performance appears primarily attributable to chat template incompatibility. Since Darwin1_v2 lacks a native chat template as a LLaMA-based model, we implemented a basic LLaMA-style template for experimentation. However, extensive error analysis reveals the model consistently failed to interpret structured prompt elements, particularly evident in its frequent disregard for <answer> tags and inability to process hybrid RAG instructions. This fundamental template mismatch suggests that simply adopting architecture-appropriate formatting schemes may be insufficient for domain-specific models to adapt to complex, retrieval-augmented workflows.

4.5 Limitations and Future Improvement Directions

4.5.1 Current Limitations

The study reveals two critical limitations including hardware constraints and chat template incompatibility:

(1) Hardware Constraints:

Due to GPU memory limitations, the current implementation fails to load two different local models at the same time (e.g., Darwin1_v2 as the main model and Qwen2.5-7B-Instruct as the format checker model), which restricts the ability to test optimal model combinations for improved accuracy. The current experiments were limited to demand the same model to be both roles, potentially missing better synergies.

(2) Inappropriate Chat Template of Darwin1_v2:

The currently adopted LLaMA-style chat template misaligns with the prompt structure we use in the experiment, resulting in the situation that the model can't fully understand queries from users and generate effective outputs. Also, Darwin model consistently fails to adhere to the required output format.

4.5.2 Future Improvements

Future work should prioritize multi-model deployment and template redesign for domain-specific models:

(1) To address the GPU memory resource shortage, we can implement memory-efficient loading techniques and select remote LLMs to identify optimal main-checker models combinations.

(2) By analyzing raw responses of Darwin1_v2 model, we can identify the common failure patterns and modify the chat template to better align with current workflow requirements. This modification is expected to improve both accuracy and format compliance, thereby reducing the performance gap with general-purpose models like Qwen.

5 CONCLUSION

The main work of this dissertation is summarized as following:

By systematically reviewing domestic and international literature on large language models (LLMs) in materials science, this paper provides a comprehensive overview of current applications of LLMs in specialized scientific domains while critically analyzing the strengths and limitations of conventional methodologies. To address the key challenges of outdated knowledge bases and inadequate simulation performance in traditional LLM-based material simulations, we propose a framework integrating LLMs with Retrieval-Augmented Generation (RAG) and AI agent technologies, establishing the foundational research direction for the study.

As the theoretical basis, this paper briefly introduced the core principles and architectural frameworks of LLM, RAG and agent, then further analyzed the feasibility and reliability of their integration for materials simulation applications. To evaluate the performance of individual LLMs on this domain, we generated a code-based and a doc-based dataset related to Pymatgen through the Gemini-2.0-flash model. Thirteen representative LLMs, including both general-purposed and domain-specific models, were selected to for performance evaluation on these datasets using properly designed prompts. The Benchmark results demonstrate that general-purpose models significantly outperformed domain-specific models, owing to their superior natural language comprehension and instruction-following capabilities developed through broad pretraining.

To further improve the abilities of LLMs in materials simulation, we proposed an enhanced framework integrating RAG and format validation. This framework addresses LLMs' knowledge limitations in materials science via dynamic retrieval augmentation and guarantees output reliability through strict format validation. With RAG enhancement, the Qwen2.5-7B-Instruct model demonstrates measurable performance gains on both datasets (+2.73% on code-based tasks and +7.13% on doc-based tasks), while Darwin1_v2 maintains consistently poor performance due to architectural incompatibilities. The experimental results demonstrate the effectiveness of the RAG-enhanced framework for materials simulation applications while revealing several limitations that require attention in future implementations. Notably, the

framework’s modular architecture shows significant potential for broader adaptation across diverse scientific domains.

Due to time limitation, certain aspects of this study require further improvement and refinement in future research: First, the configuration of domain-specific models (e.g., the chat template of Darwin1_v2) requires optimization to enhance their compatibility with the experimental framework; Second, a more comprehensive evaluation of diverse LLMs is necessary to determine the most effective pairing for the main model and format validation agent.

REFERENCES

[1] S. Minaee et al., “Large language models: A survey,” Mar. 23, 2025, arXiv: arXiv:2402.06196. doi: 10.48550/arXiv.2402.06196.

[2] A. Singh, "Exploring Language Models: A Comprehensive Survey and Analysis," 2023 International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE), Chennai, India, 2023, pp. 1-4, doi: 10.1109/RMKMATE59243.2023.10369423.

[3] L. Ouyang et al., “Training language models to follow instructions with human feedback,” Mar. 04, 2022, arXiv: arXiv:2203.02155. doi: 10.48550/arXiv.2203.02155.

[4] OpenAI et al., “GPT-4 technical report,” Mar. 04, 2024, arXiv: arXiv:2303.08774. doi: 10.48550/arXiv.2303.08774.

[5] J. Wei et al., “Emergent abilities of large language models,” Oct. 26, 2022, arXiv: arXiv:2206.07682. doi: 10.48550/arXiv.2206.07682.

[6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” Oct. 10, 2023, arXiv: arXiv:2303.11366. doi: 10.48550/arXiv.2303.11366.

[7] G. Wang et al., “Voyager: An open-ended embodied agent with large language models,” Oct. 19, 2023, arXiv: arXiv:2305.16291. doi: 10.48550/arXiv.2305.16291.

[8] J. Wei et al., “Emergent abilities of large language models,” Oct. 26, 2022, arXiv: arXiv:2206.07682. doi: 10.48550/arXiv.2206.07682.

[9] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” Mar. 10, 2023, arXiv: arXiv:2210.03629. doi: 10.48550/arXiv.2210.03629.

[10] R. Nakano et al., “WebGPT: Browser-assisted question-answering with human feedback,” Jun. 01, 2022, arXiv: arXiv:2112.09332. doi: 10.48550/arXiv.2112.09332.

[11] J. Wei et al., “Finetuned language models are zero-shot learners,” Feb. 08, 2022, arXiv: arXiv:2109.01652. doi: 10.48550/arXiv.2109.01652.

[12] V. Sanh et al., “Multitask prompted training enables zero-shot task generalization,” Mar. 17, 2022, arXiv: arXiv:2110.08207. doi: 10.48550/arXiv.2110.08207.

[13] T. B. Brown et al., “Language models are few-shot learners,” Jul. 22, 2020, arXiv: arXiv:2005.14165. doi: 10.48550/arXiv.2005.14165.

[14] L. Wang et al., “A survey on large language model based autonomous agents,” Front. Comput. Sci., vol. 18, no. 6, p. 186345, Dec. 2024, doi: 10.1007/s11704-024-40231-1.

[15] Z. He et al., “ChatEDA: A large language model powered autonomous agent for EDA,” IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 43, no. 10, pp. 3184–3197, Oct. 2024, doi: 10.1109/TCAD.2024.3383347.

[16] S. S. Chong, Y. S. Ng, H.-Q. Wang, and J.-C. Zheng, “Advances of machine learning in materials science: Ideas and techniques,” Front. Phys., vol. 19, no. 1, p. 13501, Feb. 2024, doi: 10.1007/s11467-023-1325-z.

[17] T. Weiss, E. Mayo Yanes, S. Chakraborty, L. Cosmo, A. M. Bronstein, and R. Gershoni-Poranne, “Guided diffusion for inverse molecular design,” Nat Comput Sci, vol. 3, no. 10, pp. 873–882, Oct. 2023, doi: 10.1038/s43588-023-00532-0.

[18] S. S. Chong, Y. S. Ng, H.-Q. Wang, and J.-C. Zheng, “Advances of machine learning in materials science: Ideas and techniques,” Front. Phys., vol. 19, no. 1, p. 13501, Feb. 2024, doi: 10.1007/s11467-023-1325-z.

[19] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, “Agentic retrieval-augmented generation: A survey on agentic RAG,” Feb. 04, 2025, arXiv: arXiv:2501.09136. doi: 10.48550/arXiv.2501.09136.

[20] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” Apr. 12, 2021, arXiv: arXiv:2005.11401. doi: 10.48550/arXiv.2005.11401.

[21] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-augmented language model pre-training,” Feb. 10, 2020, arXiv: arXiv:2002.08909. doi: 10.48550/arXiv.2002.08909.

[22] S. Borgeaud et al., “Improving language models by retrieving from trillions of tokens,” Feb. 07, 2022, arXiv: arXiv:2112.04426. doi: 10.48550/arXiv.2112.04426.

[23] B. Wang et al., “Shall we pretrain autoregressive language models with retrieval? A comprehensive study,” Dec. 21, 2023, arXiv: arXiv:2304.06762. doi: 10.48550/arXiv.2304.06762.

[24] Z. Yu, C. Xiong, S. Yu, and Z. Liu, “Augmentation-adapted retriever improves generalization of language models as generic plug-in,” May 27, 2023, arXiv: arXiv:2305.17331. doi: 10.48550/arXiv.2305.17331.

[25] D. Cheng et al., “UPRISE: Universal prompt retrieval for improving zero-shot evaluation,” Dec. 16, 2023, arXiv: arXiv:2303.08518. doi: 10.48550/arXiv.2303.08518.

[26] X. Li et al., “Structure-aware language model pretraining improves dense retrieval on structured data,” May 31, 2023, arXiv: arXiv:2305.19912. doi: 10.48550/arXiv.2305.19912.

[27] Z. Guo, S. Cheng, Y. Wang, P. Li, and Y. Liu, “Prompt-guided retrieval augmentation for non-knowledge-intensive tasks,” May 28, 2023, arXiv: arXiv:2305.17653. doi: 10.48550/arXiv.2305.17653.

[28] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, “Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy,” Oct. 23, 2023, arXiv: arXiv:2305.15294. doi: 10.48550/arXiv.2305.15294.

[29] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Toronto, Canada: Association for Computational Linguistics, 2023, pp. 10014–10037. doi: 10.18653/v1/2023.acl-long.557.

[30] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” Jan. 10, 2023, arXiv: arXiv:2201.11903. doi: 10.48550/arXiv.2201.11903.

[31] Q. Long et al., “Autonomous planning and processing framework for complex tasks based on large language models,” Zidonghua Xuebao, vol. 50, no. 4, pp. 1-11, Apr. 2024. [Chinese]